CS 203: Software Tools & Techniques for AI

Team- 06

Swayam Giridhar Borate (23110066)

Aditya Prakash Borate (23110065)

Github repo link: https://github.com/adi776borate/CS203-STT-Labs/tree/main/CS203_Lab_06

SECTION - 1

- Code for preprocessing the Iris dataset by normalizing features, one-hot encoding labels, and stratified splitting into training (70%), validation (10%), and test (20%) sets while ensuring class balance.

```
[15] # Load and preprocess Iris dataset
     iris = load_iris()
     X = iris.data
     y = iris.target.reshape(-1, 1)  # Reshape for one-hot encoding
     X.shape, y.shape

     ((150, 4), (150, 1))

[16] # One-hot encode labels
     encoder = OneHotEncoder(sparse_output=False)
     y = encoder.fit_transform(y)

[17] # Normalize features to [0,1]
     scaler = MinMaxScaler()
     X = scaler.fit_transform(X)

[18] # Split dataset with stratification into train (70%)
     X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)

     # Further split into validation (10%) & test (20%) from remaining 30%
     X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=2/3, stratify=y_temp, random_state=42)

[19] # Check the no. of sample of each class in train, val, and test

     print("Training data:")
     unique_train, counts_train = np.unique(np.argmax(y_train, axis=1), return_counts=True)
     for label, count in zip(unique_train, counts_train):
         print(f"Class {label}: {count} samples")

     print("\nValidation data:")
     unique_val, counts_val = np.unique(np.argmax(y_val, axis=1), return_counts=True)
     for label, count in zip(unique_val, counts_val):
         print(f"Class {label}: {count} samples")

     print("\nTest data:")
     unique_test, counts_test = np.unique(np.argmax(y_test, axis=1), return_counts=True)
     for label, count in zip(unique_test, counts_test):
         print(f"Class {label}: {count} samples")
```

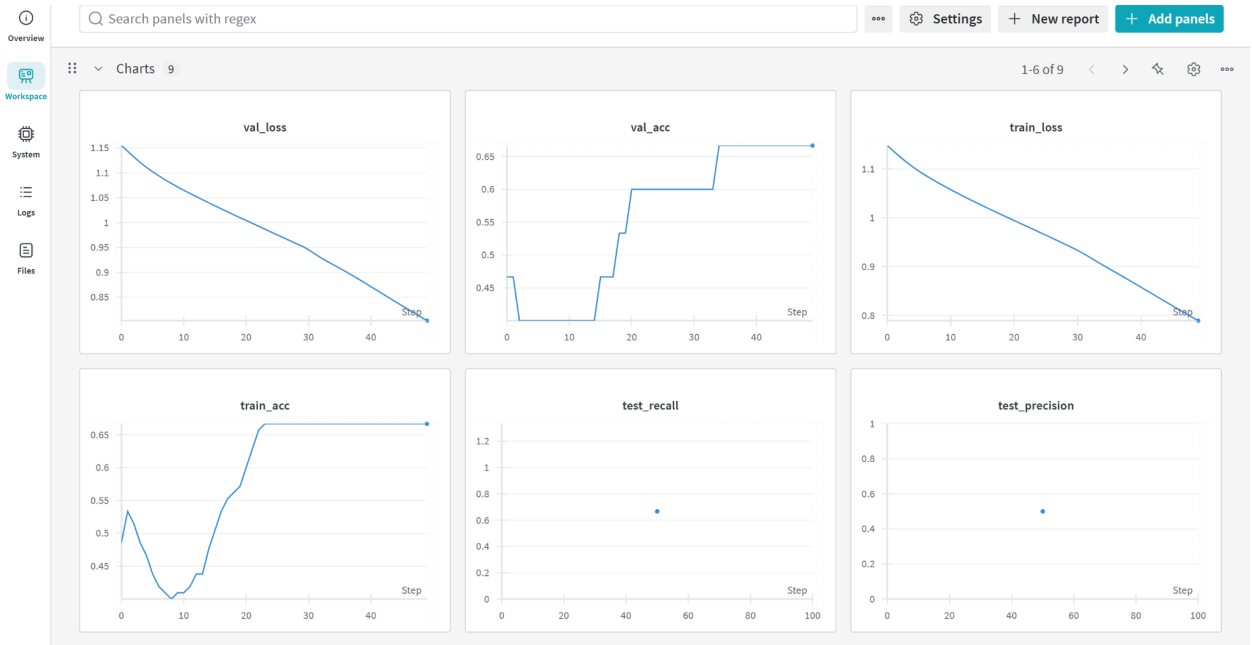
- Function for Training

```python
# Training Loop
def train(train_dataset, val_dataset, model, optimizer, train_acc_metric, val_acc_metric, epochs=10):
    train_losses, val_losses = [], []

    for epoch in range(epochs):
        print(f"\nEpoch {epoch+1}/{epochs}")

        train_loss = []
        val_loss = []

        # Training Loop
        for x_batch_train, y_batch_train in train_dataset:
            loss_value = train_step(x_batch_train, y_batch_train, model, optimizer, loss_fn, train_acc_metric)
            train_loss.append(float(loss_value))

        # Validation Loop
        for x_batch_val, y_batch_val in val_dataset:
            val_loss_value = test_step(x_batch_val, y_batch_val, model, loss_fn, val_acc_metric)
            val_loss.append(float(val_loss_value))

        # Compute Metrics
        train_acc = train_acc_metric.result()
        val_acc = val_acc_metric.result()

        # Store losses for plotting
        train_losses.append(np.mean(train_loss))
        val_losses.append(np.mean(val_loss))

        print(f"Train Loss: {train_losses[-1]:.4f} - Train Acc: {train_acc:.4f} - Val Loss: {val_losses[-1]:.4f} - Val Acc: {val_acc:.4f}")

        # Reset metrics
        train_acc_metric.reset_state()
        val_acc_metric.reset_state()

        # Log metrics with W&B
        wandb.log({"epoch": epoch+1,
                   "train_loss": train_losses[-1],
                   "train_acc": float(train_acc),
                   "val_loss": val_losses[-1],
                   "val_acc": float(val_acc)})

    return train_losses, val_losses
```

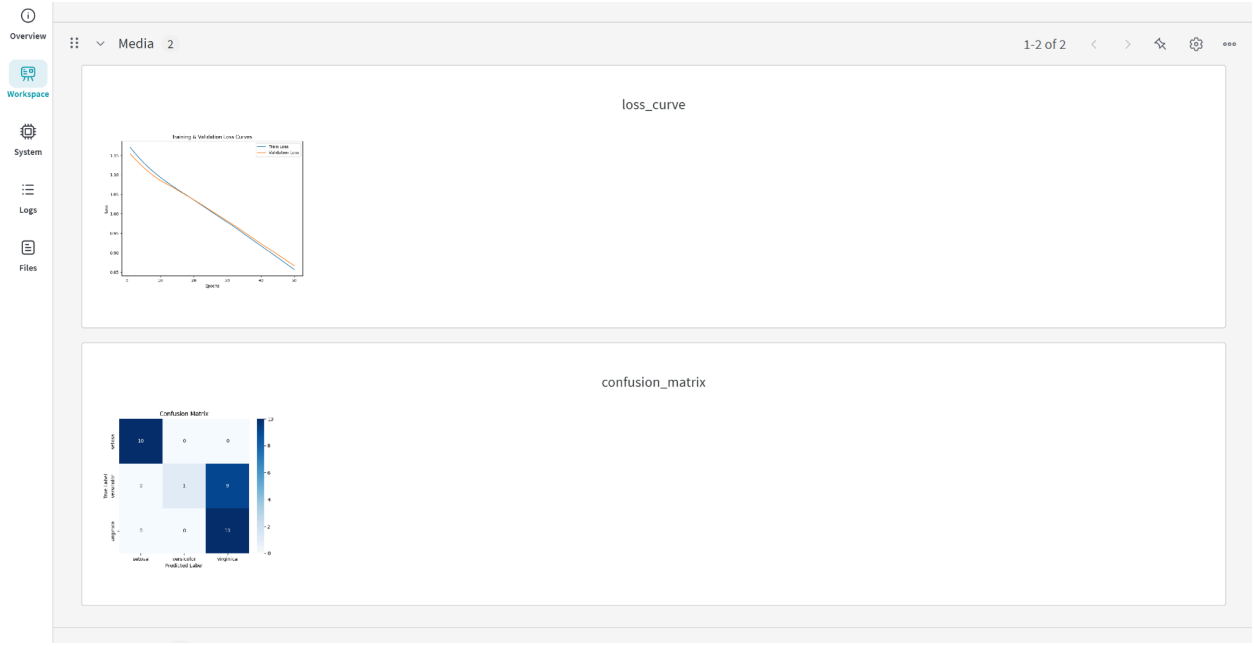
- Code for Training and Logging with Weights & Biases (W&B)

```python
# Initialize Weights & Biases (W&B)
config = {
    "learning_rate": 0.001,
    "epochs": 50,
    "batch_size": 32,
    "architecture": "MLP",
    "dataset": "Iris",
    "layers": ["Dense(16, ReLU)", "Dense(3, Softmax)"]
}

run = wandb.init(project="MLP-IRIS", config=config)
config = wandb.config

# Create model, optimizer, and loss function
model = make_model()

# Define optimizer, loss, and metrics
optimizer = keras.optimizers.Adam(learning_rate=config.learning_rate)
loss_fn = keras.losses.CategoricalCrossentropy()
train_acc_metric = keras.metrics.CategoricalAccuracy()
val_acc_metric = keras.metrics.CategoricalAccuracy()

# Start Training
train_losses, val_losses = train(train_dataset, val_dataset, model, optimizer, train_acc_metric, val_acc_metric, epochs=config.epochs)

# Evaluate Model
print("==="*15)
evaluate(model, test_dataset)
print("==="*15)

# Plot Loss Curves
plt.figure(figsize=(8, 6))
plt.plot(range(1, config.epochs+1), train_losses, label="Train Loss")
plt.plot(range(1, config.epochs+1), val_losses, label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training & Validation Loss Curves")
plt.legend()
wandb.log({"loss_curve": wandb.Image(plt.gcf())}) # Log to W&B first
plt.show()
plt.close()

# Finish W&B Run
wandb.finish()
```

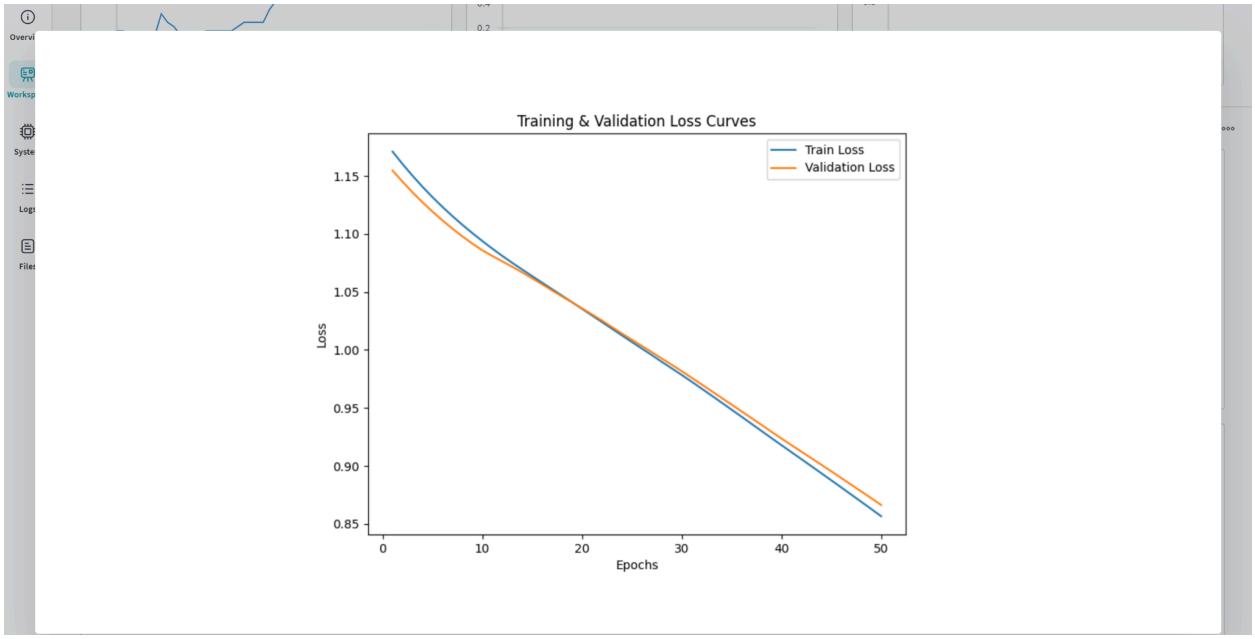
- Training & validation loss and metrics curves



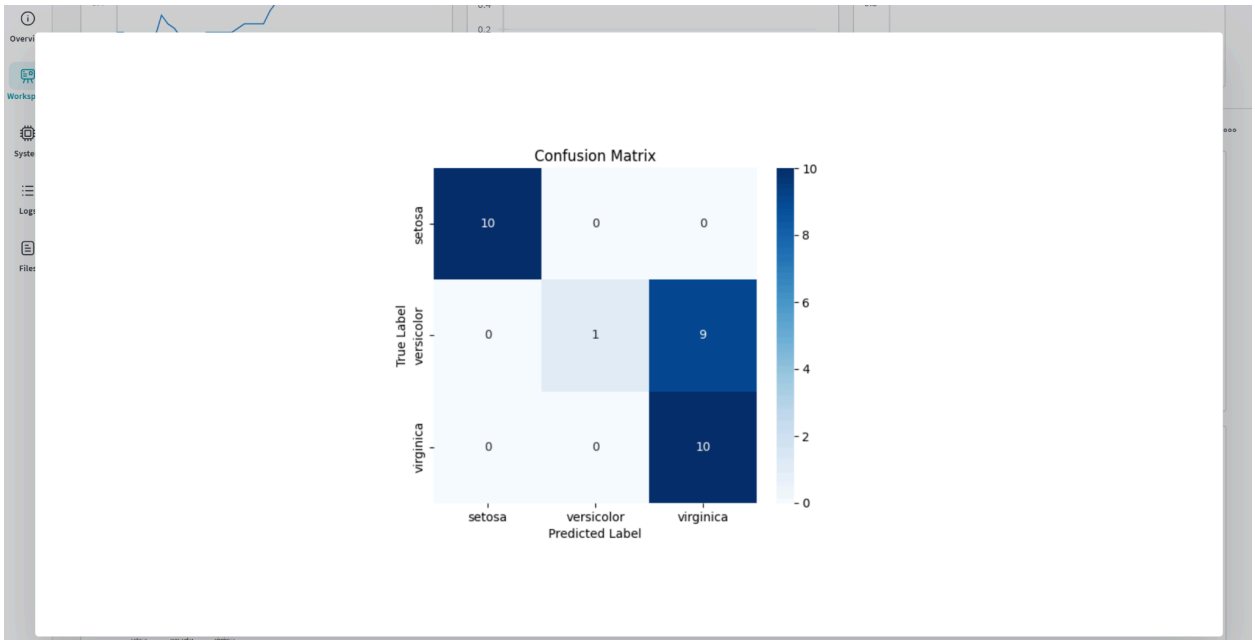
- Training & Validation loss curves and Confusion matrix logged



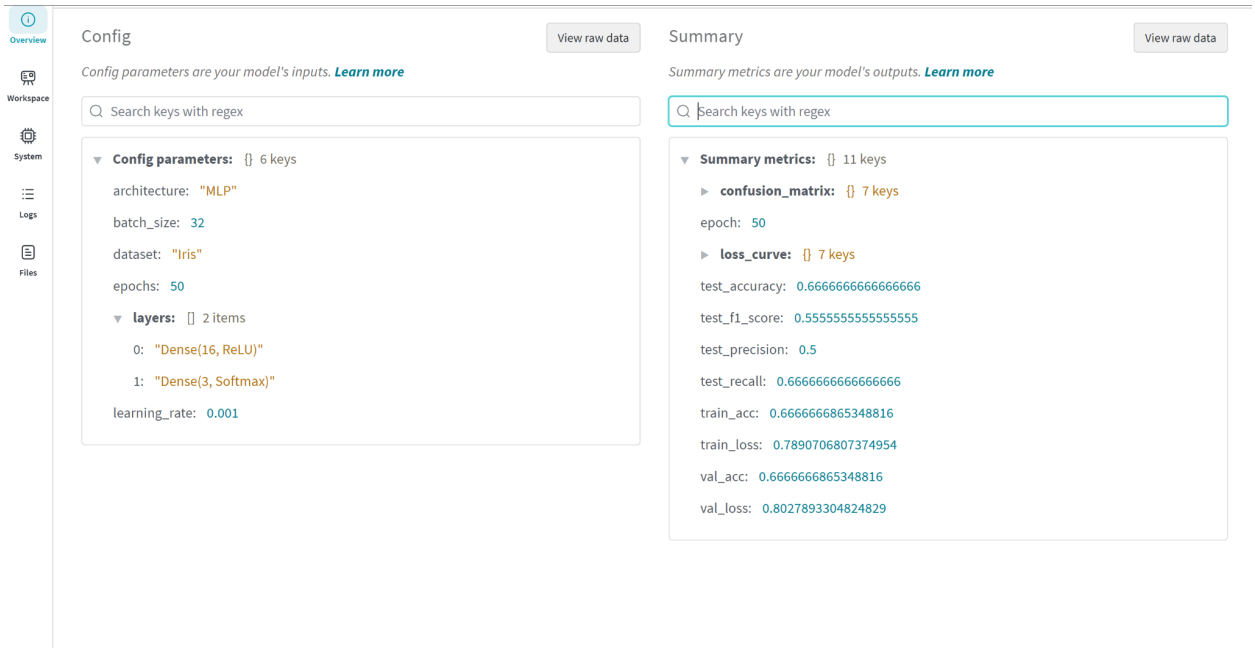
- Training & Validation Loss Curves



- Confusion matrix



- Model architecture \ Logged metrics \ Hyperparameters



- Run history and summary

Run history:

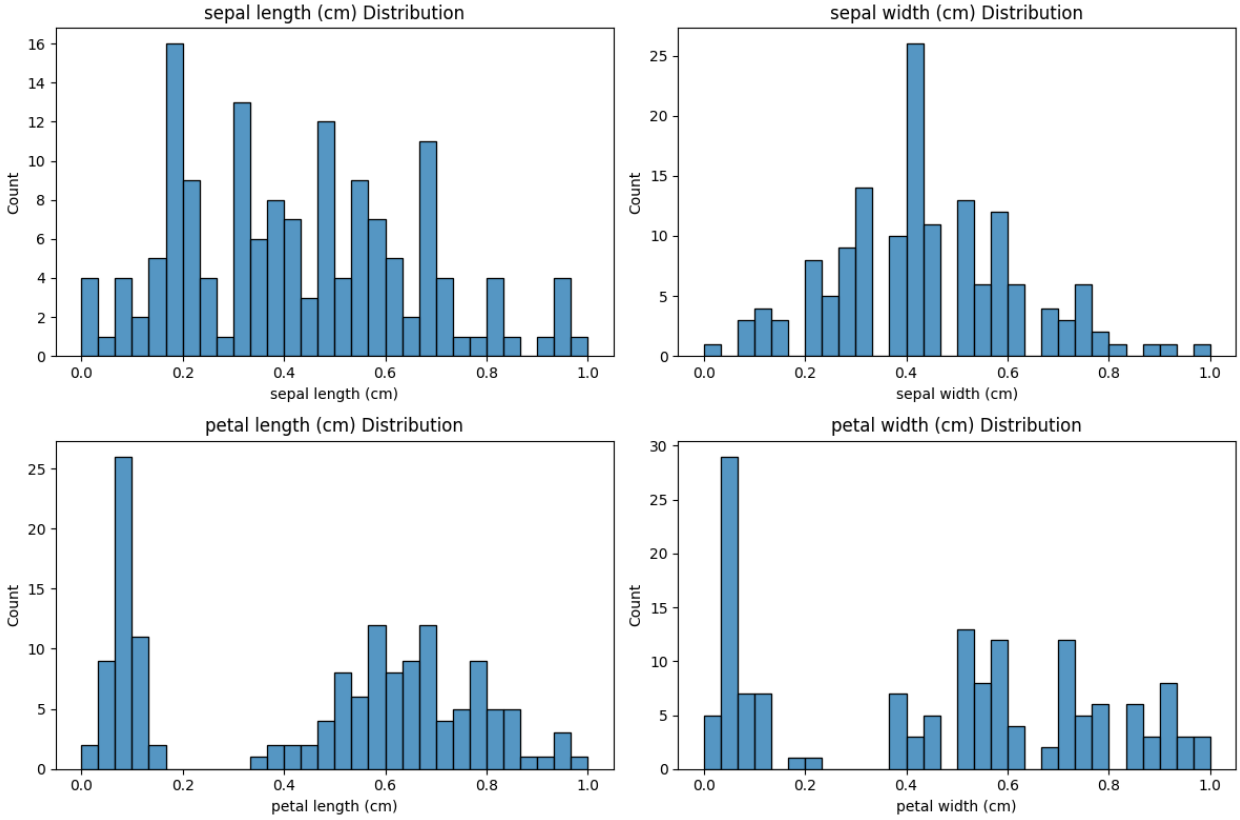
Run summary:

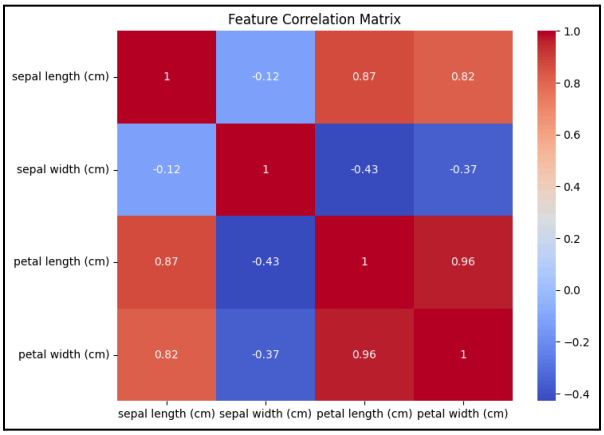
epoch	50
test_accuracy	0.66667
test_f1_score	0.55556
test_precision	0.5
test_recall	0.66667
train_acc	0.66667
train_loss	0.78907
val_acc	0.66667
val_loss	0.80279

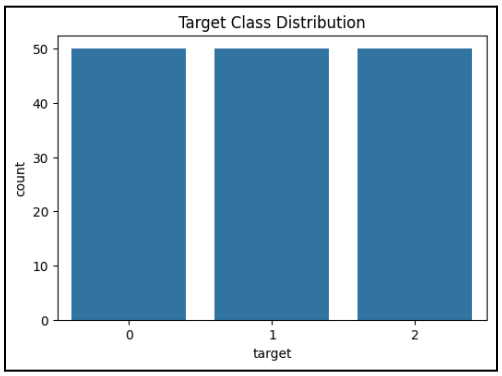
View run **run1** at: https://wandb.ai/23110065-indian-institute-of-technology-gandhinagar/MLP-IRIS/runs/yyw79x2a
View project at: https://wandb.ai/23110065-indian-institute-of-technology-gandhinagar/MLP-IRIS
Synced 5 W&B file(s), 2 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20250227_104140-yyw79x2a/logs`

Section 2

- **EDA on IRIS Data:**
  Distribution of features







Task 1: Hyperparameter Optimization

# Hyperparameters
batch_sizes = [2, 4]
learning_rates = [1e-3, 1e-5]
epochs_list = [1, 3, 5]

- Manual Tuning of hyperparameter results:

Hyperparameter Optimization Results:

	Batch Size	Learning Rate	Epochs	Accuracy	F1 Score
0	2	0.00100	1	0.766667	0.734093
1	2	0.00100	3	1.000000	1.000000
2	2	0.00100	5	1.000000	1.000000
3	2	0.00001	1	0.433333	0.346667
4	2	0.00001	3	0.566667	0.474250
5	2	0.00001	5	0.566667	0.476530
6	4	0.00100	1	0.733333	0.675485
7	4	0.00100	3	1.000000	1.000000
8	4	0.00100	5	1.000000	1.000000
9	4	0.00001	1	0.366667	0.280193
10	4	0.00001	3	0.533333	0.439394
11	4	0.00001	5	0.566667	0.474250

Observations:

1. **Effect of Epochs:**
    - For a higher learning rate (0.001), increasing the number of epochs improves accuracy and F1 score significantly.
    - With just 1 epoch, the model achieves **76.67% accuracy** with a batch size of 2, but with 3 or more epochs, it reaches **100% accuracy and an F1 score of 1.00** consistently.
    - This indicates that training for at least **3 epochs is sufficient** for the model to fully learn the dataset at this learning rate.
2. **Effect of Learning Rate:**
    - A high learning rate (**0.001**) results in quick convergence, achieving perfect accuracy within **3 to 5 epochs**, regardless of batch size.
    - A low learning rate (**0.00001**) leads to much poorer performance, with accuracy ranging from **36.67% to 56.67%**, even after 5 epochs.

- This suggests that **0.00001 is too small**, causing slow learning and ineffective training.

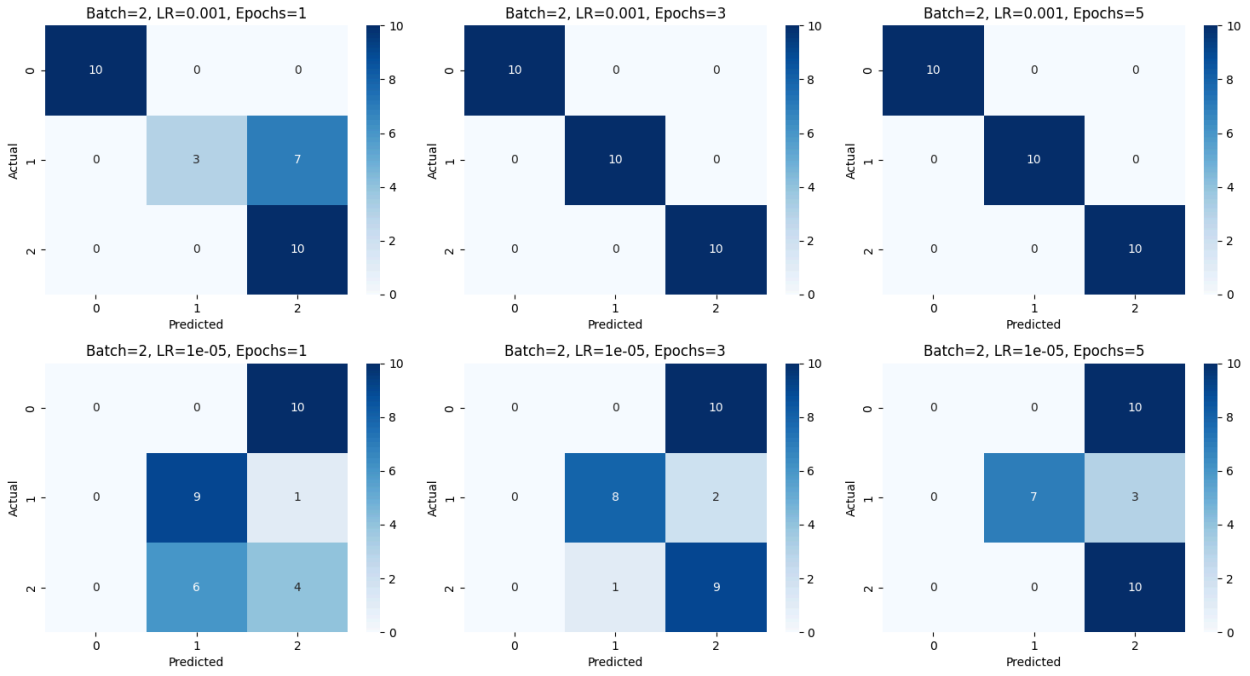
3. **Effect of Batch Size:**

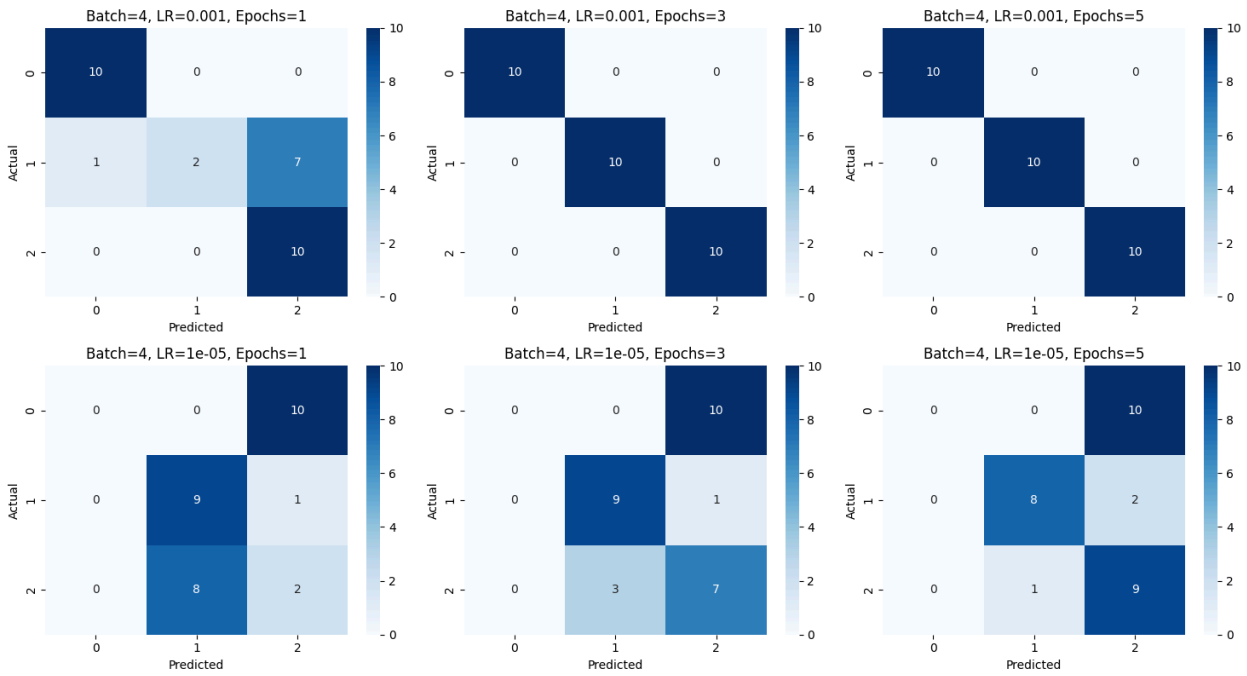
   - For **batch size = 2**, performance improves significantly with epochs when the learning rate is **0.001**.
   - For **batch size = 4**, a similar trend is observed, where the model reaches **100% accuracy within 3 epochs at a learning rate of 0.001**.
   - However, at a very low learning rate (**0.00001**), batch size changes do not significantly affect performance, as the model struggles to learn.

Conclusions:

- **Best Hyperparameter Combination:**
  - **Batch size = 2 or 4, Learning rate = 0.001, Epochs ≥ 3** → Leads to **100% accuracy and F1 score of 1.00**.
- **Worst Performing Setting:**
  - **Batch size = 4, Learning rate = 0.00001, 1 epoch** → Results in **only 36.67% accuracy and a poor F1 score of 0.28**.

- Confusion matrix of all combinations of hyperparameters





- Show the inputs, prediction, and truth values for five samples from the test set:

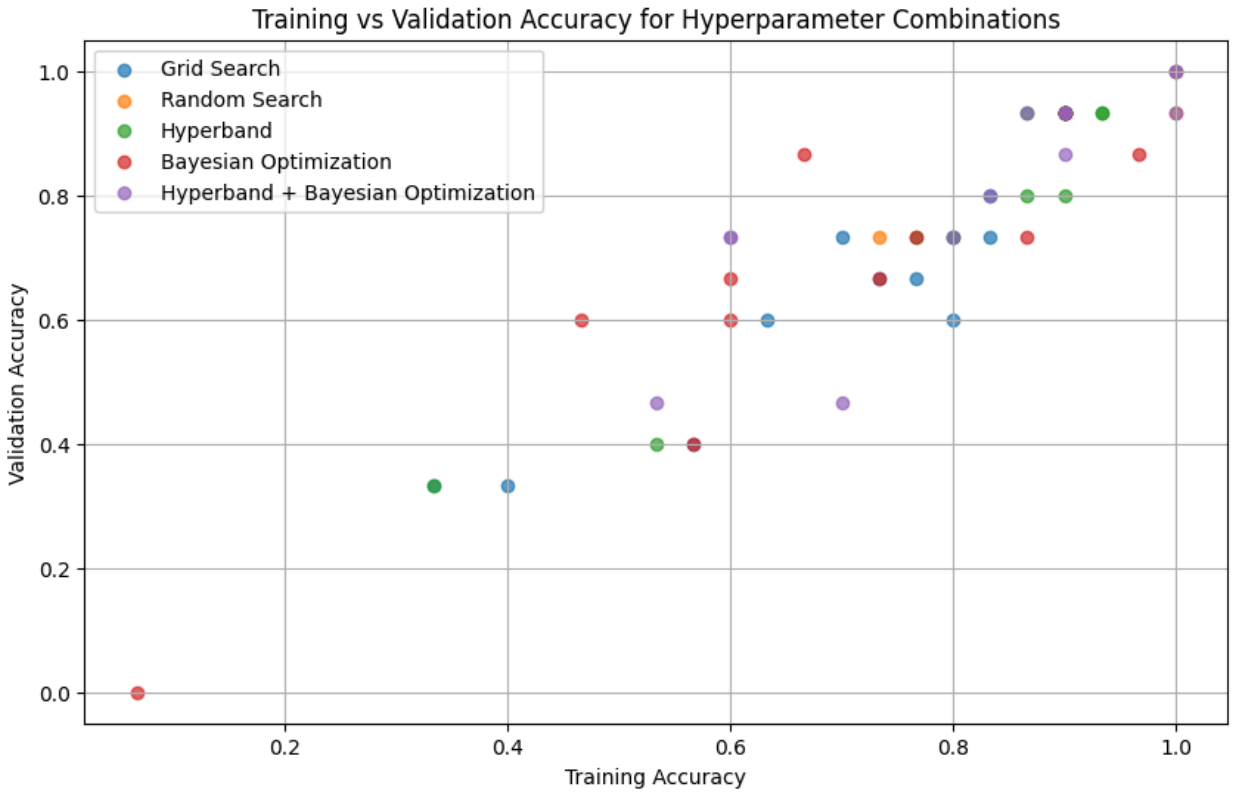
```
Using best hyperparameters: Batch=2, LR=0.001, Epochs=3

Sample Predictions:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	predicted
27	0.027778	0.375000	0.067797	0.041667	0	0
15	0.638889	0.375000	0.610169	0.500000	1	1
23	0.777778	0.416667	0.830508	0.833333	2	2
17	0.333333	0.125000	0.508475	0.500000	1	1
8	0.194444	0.583333	0.084746	0.041667	0	0

- Describe the performance for each hyperparameter combination over accuracy and F1.
  - **Epochs:** Performance (Accuracy & F1 Score) improves as the number of epochs increases. For both batch sizes, training with 3 or 5 epochs consistently reaches perfect scores (1.00), while 1 epoch results in lower accuracy.
  - **Learning Rate:** A higher learning rate (0.001) yields significantly better performance, achieving perfect scores with sufficient epochs. In contrast, a lower learning rate (0.00001) leads to poor accuracy, especially for fewer epochs.
  - **Batch Size:** A smaller batch size (2) generally performs better than a larger batch size (4). For lower learning rates, performance is slightly better with batch size 2, while at higher learning rates, both batch sizes reach optimal performance with more epochs.

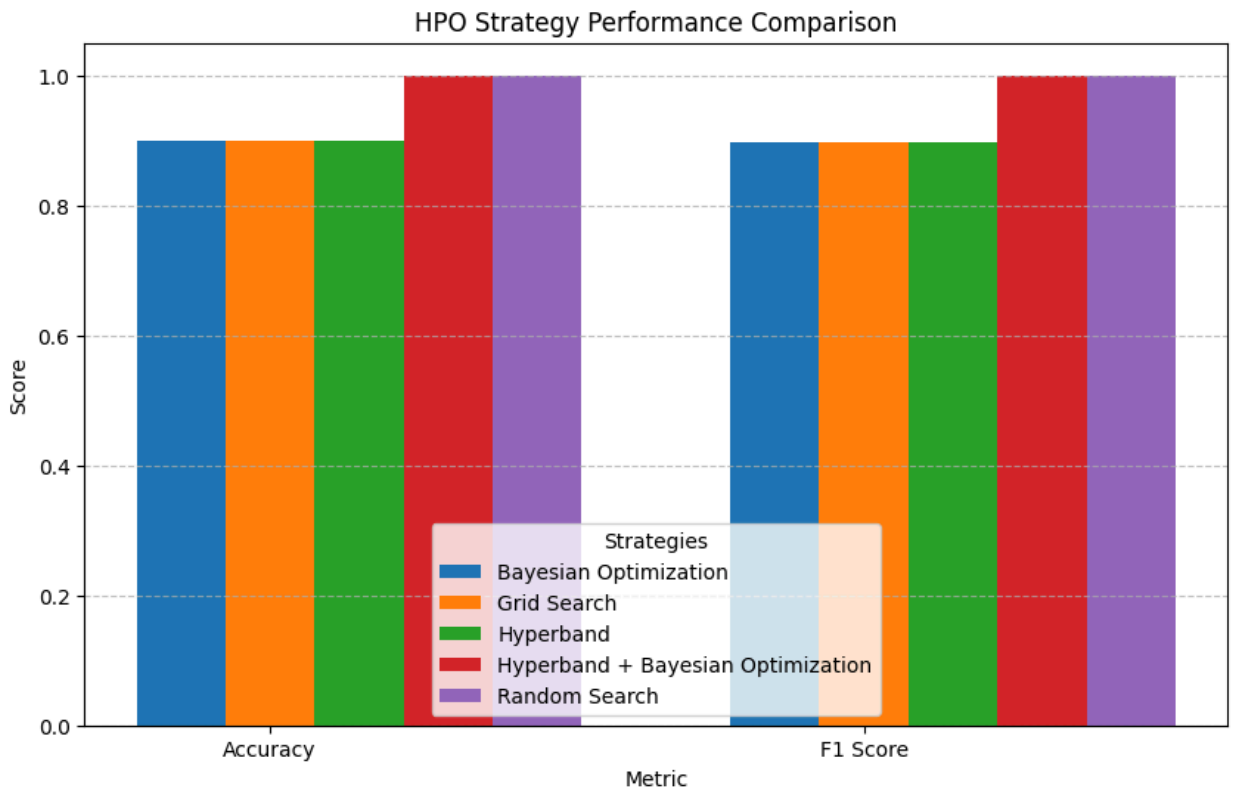
Task 2: Automated Hyperparameter Search

- Scatter Plot of Training and Validation Accuracy for Each Strategy



- Hyperparameter Optimization Strategy Performance Comparison

Strategy	Accuracy	F1 Score
Bayesian Optimization	0.9	0.897698
Grid Search	0.9	0.897698
Hyperband	0.9	0.897698
Hyperband + Bayesian Optimization	1.0	1.000000
Random Search	1.0	1.000000



Conclusion:

- Which approach is better and why?

  Automated tuning consistently outperforms manual search in terms of consistency and efficiency. Hyperband + Bayesian Optimization achieve perfect accuracy (1.00), while Random Search (1.00) outperforms some manual trials, but this is not always true. Grid Search, Bayesian, Hyperband all are performing equally well (0.9). This is due to small dataset size. Automation reduces trial-and-error, making it a more reliable and faster approach.

  So, the best strategy is Hyperband + Bayesian Optimization with Accuracy = 1.0 and F-1 score = 1.0.